

Sprint Documentation

Sprint 14	Start Date: 25/03/2026	Final Date: 30/03/2026
Projetc Name:	civitas-backend	
Ano: 5º	Análise e Desenvolvimento de Sistemas - AMS	
Team Members		
Carlos Alberto Ferreira Candido		
João Antonio Gomes Cestonaro		
Giovanni da Silva Nascimento		

Sprint Backlog

Task #	Description	Start Date	Final date
#64	Validações dos campos da despesa	25/03/2026	30/03/2026

Task Description

Task #	Description	Assigned To	Status	Estimated Hours	Logged Hours
#64	Garantir que o cadastro de despesas possua validações completas de integridade, consistência e regras de negócio, evitando pagamentos duplicados, dados inválidos e inconsistências financeiras no sistema.	Carlos Alberto Ferreira Candido, João Antonio Gomes Cestonaro, Giovanni da Silva Nascimento	Completo	15 horas	15 horas

Sprint Description

📌 Tarefa: Validações de Cadastro de Despesa

🎯 Objetivo

Garantir que o cadastro de despesas possua validações completas de integridade, consistência e regras de negócio, evitando pagamentos duplicados, dados inválidos e inconsistências financeiras no sistema.

📖 Descrição

Implementar validações no backend para os campos do cadastro de despesa, assegurando:

- Dados obrigatórios

- Formatação correta
 - Consistência de valores
 - Regras de negócio específicas
 - Integridade com entidades relacionadas
-

☑ Critérios de Aceite

◆ Campos Obrigatórios

- - Validar que os campos obrigatórios não estão vazios:
 - numeroDocumento
 - dataEmissao
 - dataVencimento
 - situacao
 - idTipoDespesa
 - idOrcamento
 - idInstituicao
 - idFornecedor
 - idUsuario
-

◆ Validação de Número do Documento

- Deve possuir no máximo 100 caracteres

Não pode ser vazio

Deve ser único no sistema (evitar pagamento duplicado)

Remover espaços em branco no início/fim (trim)

◆ Validação de UC (Unidade Consumidora)

- Deve possuir no máximo 100 caracteres

Campo opcional por padrão

- Tornar obrigatório quando:
 - TipoDespesa.SolicitaUc = true

Remover espaços em branco (trim)

◆ Validação de Data de Emissão

- Deve ser uma data válida

Não pode ser futura

Padronizar formato (preferencialmente DateOnly)

◆ Validação de Data de Vencimento

- Deve ser uma data válida

Não pode ser anterior à data de emissão

◆ Validação de Consumo Previsto

- Deve ser um número válido

Não pode ser negativo

Pode ser zero (dependendo do tipo de despesa)

◆ Validação de Situação

-
- Aceitar apenas valores válidos:
 - 1 = Ativo
 - 2 = Inativo

Utilizar enum para validação

◆ Validação de Relacionamentos (Foreign Keys)

- - Validar que os IDs são válidos (> 0):
 - idTipoDespesa
 - idOrcamento
 - idInstituicao
 - idFornecedor
 - idUsuario
 - Validar existência no banco:
 - TipoDespesa deve existir
 - Orçamento deve existir
 - Instituição deve existir
 - Fornecedor deve existir
 - Usuário deve existir
-

◆ Regras de Negócio

✎ UC obrigatória por TipoDespesa

-
- Se TipoDespesa.SolicitaUc = true:

- UC deve ser obrigatória
-

Controle de duplicidade

- - Não permitir cadastro de despesa com:
 - Mesmo numeroDocumento para o mesmo fornecedor
-

Consistência com Orçamento

- Validar se a despesa está vinculada a um orçamento válido
-

Integridade da Instituição

- Validar se a despesa pertence a uma instituição válida
-

Normalização de Dados

- - Aplicar trim em campos string:
 - numeroDocumento
 - uc

Padronizar armazenamento de datas

Observações e Sugestões

- Utilizar DataAnnotations para validações básicas
- Implementar validações mais complexas na camada Service
- Verifique o trabalho antes do envio

Sprint Results

Primeiramente foram feitos métodos no service para validar relacionamento.

3 referências

```
public async Task ValidarCadastroAsync(DespesaDTO entityDTO, int? id = null)
{
    if (entityDTO is null)
    {
        throw new ArgumentException("Dados da despesa sao obrigatorios.");
    }

    if (id.HasValue && id.Value <= 0)
    {
        throw new ArgumentException("Id da despesa invalido.");
    }

    if (id.HasValue)
    {
        var existingEntity = await _despesaRepository.GetById(id.Value);
        if (existingEntity is null)
        {
            throw new KeyNotFoundException($"Entidade com id {id.Value} nao encontrada.");
        }
    }
}
```

```
entityDTO.Id = id ?? 0;

NormalizarCampos(entityDTO);
ValidarCamposBasicos(entityDTO);
await ValidarRelacionamentosAsync(entityDTO);
await ValidarUnicidadeAsync(entityDTO, id);
}
```

1 referência

```
private async Task ValidarRelacionamentosAsync(DespesaDTO despesaDTO)
{
    var tipoDespesa = await _tipoDespesaRepository.GetById(despesaDTO.IdTipoDespesa);
    if (tipoDespesa is null)
    {
        throw new ArgumentException("TipoDespesa informado nao foi encontrado.");
    }

    if (tipoDespesa.SolicitaUc == SolicitaUc.Sim && string.IsNullOrEmpty(despesaDTO.UC))
    {
        throw new ArgumentException("UC e obrigatoria para o tipo de despesa informado.");
    }

    var orcamento = await _orcamentoRepository.GetById(despesaDTO.IdOrcamento);
```

```
    if (orcamento is null)
    {
        throw new ArgumentException("Orcamento informado nao foi encontrado.");
    }

    var instituicao = await _instituicaoRepository.GetById(despesaDTO.IdInstituicao);
    if (instituicao is null)
    {
        throw new ArgumentException("Instituicao informada nao foi encontrada.");
    }

    if (orcamento.IdInstituicao != despesaDTO.IdInstituicao)
    {
        throw new ArgumentException("O orcamento informado nao pertence a instituicao selecionada.");
    }

    if (orcamento.IdTipoDespesa > 0 && orcamento.IdTipoDespesa != despesaDTO.IdTipoDespesa)
    {
        throw new ArgumentException("O orcamento informado nao esta vinculado ao tipo de despesa selecionado.");
    }
}
```

```

var fornecedor = await _fornecedorRepository.GetById(despesaDTO.IdFornecedor);
if (fornecedor is null)
{
    throw new ArgumentException("Fornecedor informado nao foi encontrado.");
}

var usuario = await _usuarioRepository.GetById(despesaDTO.IdUsuario);
if (usuario is null)
{
    throw new ArgumentException("Usuario informado nao foi encontrado.");
}
}

private async Task ValidarUnicidadeAsync(DespesaDTO despesaDTO, int? ignoreId)
{
    var documentoDuplicado = await _despesaRepository.ExistsByNumeroDocumentoAndFornecedorAsync(
        despesaDTO.NumeroDocumento,
        despesaDTO.IdFornecedor,
        ignoreId);

```

```

if (documentoDuplicado)
{
    throw new ArgumentException("Ja existe uma despesa cadastrada para este fornecedor com o mesmo numero de documento");
}

1 referência
private static void NormalizarCampos(DespesaDTO despesaDTO)
{
    despesaDTO.NumeroDocumento = Sanitize(despesaDTO.NumeroDocumento);
    despesaDTO.UC = Sanitize(despesaDTO.UC);
    despesaDTO.DataEmissao = Sanitize(despesaDTO.DataEmissao);
}

1 referência
private static void ValidarCamposBasicos(DespesaDTO despesaDTO)
{
    if (despesaDTO.Id < 0)
    {
        throw new ArgumentException("Id da despesa nao pode ser negativo.");
    }
}

```

```

ValidarObrigatorio(despesaDTO.NumeroDocumento, nameof(despesaDTO.NumeroDocumento));
ValidarTamanhoMaximo(despesaDTO.NumeroDocumento, 100, "NumeroDocumento");
ValidarSomenteNumeros(despesaDTO.NumeroDocumento, "NumeroDocumento");
ValidarTamanhoMaximo(despesaDTO.UC, 100, "UC");
ValidarObrigatorio(despesaDTO.DataEmissao, nameof(despesaDTO.DataEmissao));

var dataEmissao = ParseDataEmissao(despesaDTO.DataEmissao);
if (dataEmissao > DateOnly.FromDateTime(DateTime.Today))
{
    throw new ArgumentException("DataEmissao nao pode ser futura.");
}

if (despesaDTO.DataVencimento == default)
{
    throw new ArgumentException("Campo obrigatorio nao preenchido: DataVencimento.");
}

if (despesaDTO.DataVencimento < dataEmissao)
{
    throw new ArgumentException("DataVencimento nao pode ser anterior a DataEmissao.");
}
}

```



```

if (double.IsNaN(despesaDTO.ConsumoPrevisto) || double.IsInfinity(despesaDTO.ConsumoPrevisto))
{
    throw new ArgumentException("ConsumoPrevisto invalido.");
}

if (despesaDTO.ConsumoPrevisto < 0)
{
    throw new ArgumentException("ConsumoPrevisto nao pode ser negativo.");
}

var valorSituacao = (int)despesaDTO.Situacao;
if (valorSituacao is not ((int)Situacao.ATIVO) and not ((int)Situacao.INATIVO))
{
    throw new ArgumentException("Situacao invalida. Valores permitidos: 1 (Ativo) ou 2 (Inativo).");
}

ValidarIdPositivo(despesaDTO.IdTipoDespesa, "IdTipoDespesa");
ValidarIdPositivo(despesaDTO.IdOrcamento, "IdOrcamento");
ValidarIdPositivo(despesaDTO.IdInstituicao, "IdInstituicao");
ValidarIdPositivo(despesaDTO.IdFornecedor, "IdFornecedor");
ValidarIdPositivo(despesaDTO.IdUsuario, "IdUsuario");

```

5 referências

```

private static void ValidarIdPositivo(int valor, string campo)
{
    if (valor <= 0)
    {
        throw new ArgumentException($"{campo} deve ser maior que zero.");
    }
}

```

1 referência

```

private static void ValidarSomenteNumeros(string valor, string campo)
{
    if (!valor.All(char.IsDigit))
    {
        throw new ArgumentException($"{campo} deve conter apenas numeros.");
    }
}

```

1 referência

```

private static DateOnly ParseDataEmissao(string dataEmissao)
{
    if (DateOnly.TryParseExact(
        dataEmissao,

```

```

        SupportedDateFormats,
        CultureInfo.InvariantCulture,
        DateTimeStyles.None,
        out var data))
    {
        return data;
    }

    if (DateOnly.TryParse(dataEmissao, PtBrCulture, DateTimeStyles.None, out data))
    {
        return data;
    }

    throw new ArgumentException("DataEmicao invalida.");
}

```

3 referências

```

private static string Sanitize(string? valor)
{
    return string.IsNullOrWhiteSpace(valor)
        ? string.Empty
        : valor.Trim();
}

```

```

    despesaDTO.DataEmicao = dataEmissao.ToString("dd-MM-yyyy", CultureInfo.InvariantCulture);
}

2 referências
private static void ValidarObrigatorio(string valor, string campo)
{
    if (string.IsNullOrEmpty(valor))
    {
        throw new ArgumentException($"Campo obrigatorio nao preenchido: {campo}.");
    }
}

2 referências
private static void ValidarTamanhoMaximo(string valor, int maximo, string campo)
{
    if (valor.Length > maximo)
    {
        throw new ArgumentException($"Campo {campo} deve conter no maximo {maximo} caracteres.");
    }
}

```

Depois foi declarado o método na interface de despesa

```

3 referências
Task ValidarCadastroAsync(DespesaDTO entityDTO, int? id = null);

```

```

1 referências
public interface IDespesaService : IGenericService<Despesa, DespesaDTO>
{
    3 referências
    Task ValidarCadastroAsync(DespesaDTO entityDTO, int? id = null);
}

```

Depois foi feito uma verificação se existe uma despesa cadastrada com o seguinte número.

```

namespace Civitas.WebAPI.Data.Interfaces
{
    4 referências
    public interface IDespesaRepository : IGenericRepository<Despesa>
    {
        2 referências
        Task<bool> ExistsByNumeroDocumentoAndFornecedorAsync(
            string numeroDocumento,
            int idFornecedor,
            int? ignoreId = null);
    }
}

```


2 referências

```
public async Task<bool> ExistsByNumeroDocumentoAndFornecedorAsync(  
    string numeroDocumento,  
    int idFornecedor,  
    int? ignoreId = null)  
{  
    var normalizedNumeroDocumento = numeroDocumento.Trim().ToUpperInvariant();  
  
    var query = _context.Despesas  
        .AsNoTracking()  
        .Where(despesa =>  
            despesa.IdFornecedor == idFornecedor &&  
            despesa.NumeroDocumento != null &&  
            despesa.NumeroDocumento.Trim().ToUpper() == normalizedNumeroDocumento);  
  
    if (ignoreId.HasValue)  
    {  
        query = query.Where(despesa => despesa.Id != ignoreId.Value);  
    }  
  
    return await query.AnyAsync();  
}
```

Depois foi alterado a relação entre instituição e despesa

```
public ICollection<Despesa> Despesas { get; set; }
```

```
modelBuilder.Entity<Despesa>()  
    .HasOne(d => d.Instituicao)  
    .WithMany(i => i.Despesas)  
    .HasForeignKey(d => d.IdInstituicao)  
    .onDelete(DeleteBehavior.Restrict);
```

Assinaturas